Microsoft Foundry

VOLT & CO / MICROSOFT FOUNDRY + CITADEL

Can we defend this release?

One customer. Two loops. Evidence at every handoff.

80 minutes of content + 10 minutes Q&A



One agent. Two loops. Four demonstrations.

Follow the lifecycle—not a catalogue of features.

00–10	**WHY + THE SYSTEM**	A trust question; two loops and two bridges
10–25	**DEMO 1 · BUILD**	Hosted agent, Toolbox, IQ, gateway and trace
25–40	**DEMO 2 · PROVE**	Business evidence, judges, controls and optimization
40–50	**DEMO 3 · RELEASE**	Candidate, case file and approval boundary
50–70	**OPERATE → DEMO 4 · IMPROVE**	A real escaped claim becomes a regression
70–90	**DECISIONS + NEXT POC + Q&A**	Ten minutes reserved for your questions

Two loops. One improving service.

Build and prove in DEV. Release and learn in production-like DEMO.

BRIDGE 1 · HANDOFF

DEV → CANDIDATE + EVIDENCE → DEMO

Code · tools · knowledge · tests · policy

INNER LOOP · BUILD & PROVE

1. **Build & connect**
   Connect tools + policy.
2. **Run, debug & trace**
   Inspect one execution.
3. **Evaluate & red-team**
   Test correctness + safety.
4. **Guard & optimize**
   Compare held-out cases.

SHARED CONTRACT & GATES

Same repo
Same tests
Same safety
Separate IDs

OUTER LOOP · RELEASE & LEARN

5. **Gate & release**
   Gate the exact candidate.
6. **Prove readiness**
   Verify the serving version.
7. **Operate & monitor**
   Watch requests + failures.
8. **Learn & improve**
   Review the next test case.

BRIDGE 2 · FEEDBACK

DEMO → HUMAN REVIEW → DEV

Reviewed traces become new regression tests.

Source workshop lifecycle diagram · adapted for synthetic DEV / DEMO

One customer. One accountable answer.

VC-10482 · Volt Charge 65 · CH-0722 · synthetic data

1

Business facts

Order, exact batch and replacement eligibility.

2

Policy evidence

An identifiable replacement policy and recall notice.

3

Responsible answer

Assessment advice—not a ticket, refund or shipment.

The harness runs the loop. We own the contract.

Start with the managed path; choose hosted when application-owned logic matters.

1

Managed harness

The platform owns orchestration and supported execution.

2

Our hosted choice

Agent Framework + Responses. Code enforces the business contract.

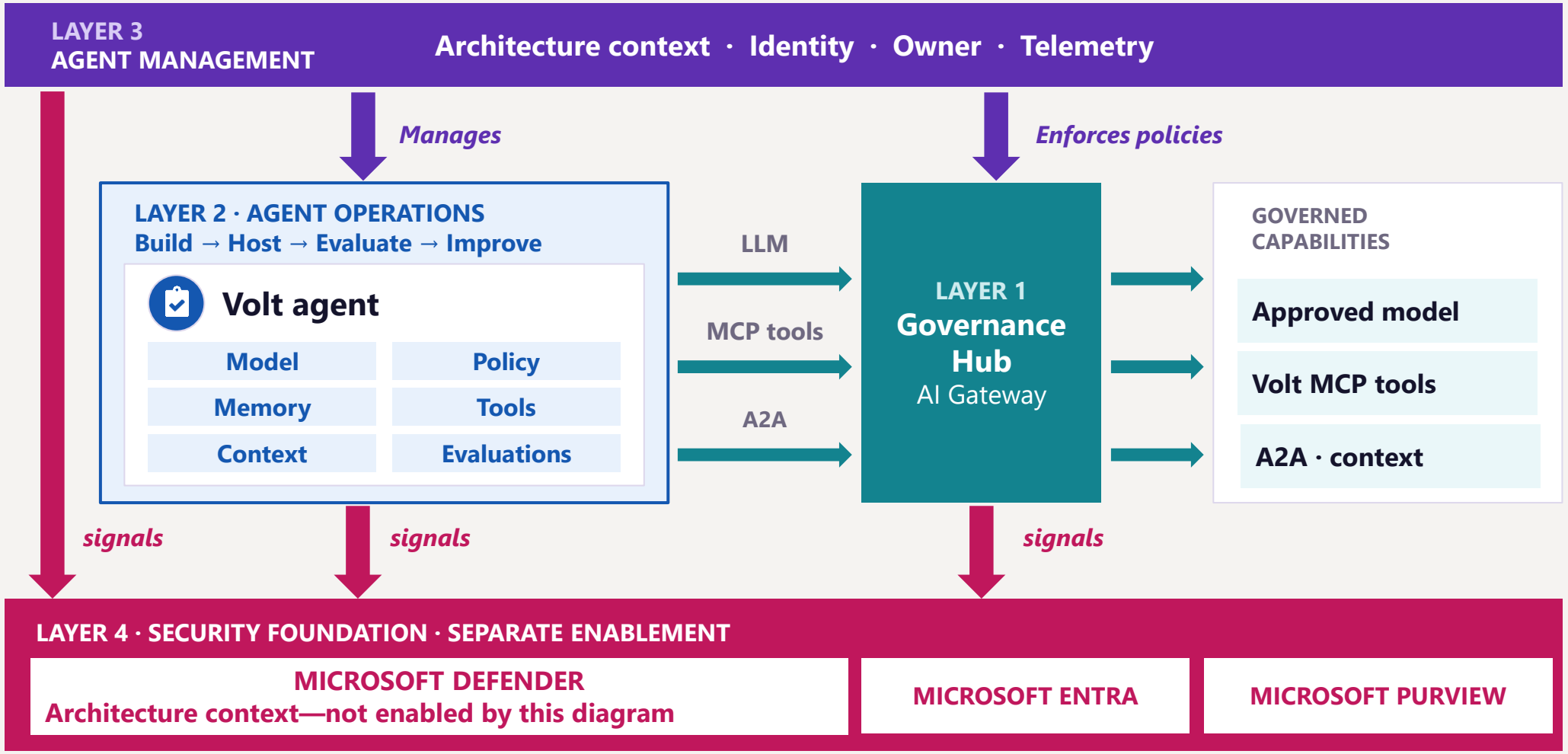
3

Copilot's role

Development assistant. The same review and release gate applies.

Where Citadel and Foundry meet

Focus today: governed model/tool access + agent operations. Other layers are architecture context.



DEMO 1 · LEAVE THE SLIDES

Build and inspect

15–25 MIN · FRESH REQUEST + IDENTIFIED TRACE

Can we explain how this answer was produced?

1. **Ask**
   Terminal · pinned released version
2. **Follow**
   Foundry · tools, IQ and actual spans
3. **Inspect**
   Two code regions, not a repository tour

RETURN TO SLIDE 08 · Debrief before the next question

A useful answer is only the beginning.

BACK FROM DEMO 1 · Configuration is not execution evidence.

1

Useful

The request reaches facts and policy.

2

Inspectable

The response identifies the executed path.

3

Still unproven

Did the answer preserve the business meaning?

Three ways to judge the same behavior

One metric cannot answer every quality question.

1	**Code**	Expected outcome + request-bound tool results + schema and references.
2	**Model rubric**	Does the prose claim an action that never happened?
3	**Human**	Review expectations, ambiguous wording and real incidents.

Name the boundary. Show its evidence.

Keep access, content safety and business correctness distinct.

1

Verified evidence

Routed utility and a recorded input-marker content_filter block.

2

Local v2 contract

Tool-derived decision and constrained no-action wording before streaming.

3

Separate activation

Tool-boundary policy and bounded red-team scan need approval and proof.

Optimization is an experiment, not a promise.

Completed prior experiment · preview · no candidate applied

1

Bounded search

Instruction-only; separate training and validation sets.

2

Actual outcome

Baseline 0.75 retained. No accepted improvement.

3

Review decision

Job success alone proves neither a quality gain nor release readiness.

DEMO 2 · LEAVE THE SLIDES

Prove the contract

31–40 MIN · NATIVE V1 HISTORY + LOCAL V2 CONTRACT

What does a passing test actually establish?

1. **Compare**
   Historical 8/9 → 9/9 was a prompt repair
2. **Challenge**
   Expected outcome AND observed evidence
3. **Enforce**
   Local Responses tests; cloud v2 pending

RETURN TO SLIDE 13 · Debrief before the next question

Bridge 1 · Candidate + evidence + approval

BACK FROM DEMO 2 · Bridge 1 carries an identifiable candidate.

1

Package once

Commit + instructions + dependencies + ZIP digest.

2

Evaluate

Fresh pinned responses; native business and calibrated action gates.

3

Approve

Protected environment; same artifact; independent DEMO verification.

A case file you can reopen

The badge is the verdict. The case file explains it.

1	**Identity**	Versioned source, package, agent, Toolbox, dataset and grader.
2	**Behavior**	Case outcomes, rubric reasons and failure evidence.
3	**Release**	Approval + promotion receipt + serving hash + fresh post-run.

DEMO 3 · LEAVE THE SLIDES

Inspect the release gate

43–50 MIN · IMPLEMENTED WORKFLOW; CLOUD ACTIVATION PENDING

Could another engineer defend this release?

1. **Open**
   Workflow and explicit activation controls
2. **Inspect**
   Actual historical case file—not v2 proof
3. **Decide**
   No live pass or approval is invented

RETURN TO SLIDE 16 · Debrief before the next question

The release passed. The contract missed something.

BACK FROM DEMO 3 · Recorded released DEMO v1

9 / 9

Historical business cases passed

“I'm escalating this to human support.”

There is no handoff tool.

The answer claims more than the tools performed.

Observe the request, not just the aggregate.

The escaped claim has a matching saved response ID and a 43-span trace.

1

Actual observation

Released DEMO response + correlated App Insights export.

2

Missing criterion

V1 checked decisions, tools and references, but missed action claims.

3

Future monitor

Sampled continuous evaluation is not yet configured.

Bridge 2 · A finding becomes a test

Operation supplies evidence. A reviewer supplies the expectation.

1

Inspect

Keep the original answer, tool evidence and version identity.

2

Annotate

No handoff occurred. Advice is not execution.

3

Regress

Add the escalation case and action rubric; show the old capture fail.

DEMO 4 · LEAVE THE SLIDES

Turn the finding into a repair

60–70 MIN · REAL RECORDED FINDING + LOCAL FAIL/FIX/PASS

What did operation teach us to test?

1. **Find**
   Same response, trace and reviewed expectation
2. **Repair**
   Copilot-assisted diff + real host-stream test
3. **Return**
   Cloud proof awaits approval

RETURN TO SLIDE 20 · Debrief before the next question

The architecture decisions we can explain

BACK FROM DEMO 4 · One reason for each architectural choice.

1	**Application**	Hosted Responses; request-bound decision renderer; read-only business API.
2	**Platform**	Toolbox + IQ; governed model and tool routes; scoped identity.
3	**Lifecycle**	Versioned tests, case file, release gate and feedback.

Your next POC: one decision, one evidence chain.

A customer-ready next step is a scoped experiment, not a platform shopping list.

1	**One decision**	Choose a business outcome and an accountable owner.
2	**One contract**	Define supported actions, expected results and failure boundaries.
3	**One proof chain**	Agree release criteria, representative cases and an operational feedback path.

Useful. Defensible. Improving.

10 MINUTES FOR QUESTIONS

1	**Own the contract**	The harness runs the loop; our application enforces business meaning.
2	**Open the evidence**	A green badge is a verdict. The case file explains it.
3	**Learn from operation**	Reviewed findings become tests before they become fixes.